

A Denotational Semantics for λ_C without Handler Parameters

Abstract

We give a denotational semantics for a fragment of the effect calculus λ_C : an expression language with algebraic effect operations, a distinguished **loss** effect used to record real-valued costs, and effect handlers of the form **with** h **from** v **handle** e that carry no separately threaded parameter. Types are interpreted by a family of *selection monads* $\widehat{S}_\epsilon(X) = (X \rightarrow \widehat{R}_\epsilon) \rightarrow \widehat{W}_\epsilon(X)$, where the auxiliary monad $\widehat{W}_\epsilon$ is obtained by applying the free monad transformer for the effect signature at level ϵ to the writer monad for a loss monoid R . Concretely, this attaches a loss contribution to *every* node of an effect tree, rather than combining all recorded loss into a single value at each leaf. Operation signatures are restricted to ground (first-order) types, which lets $\widehat{W}_\epsilon$ be defined by ordinary structural recursion. We develop the monad structure of $\widehat{W}_\epsilon$, the selection monad $\widehat{S}_\epsilon$, the semantics of expressions, and the semantics of handlers, and discuss the one significant consequence of attaching loss at every node rather than only at leaves: loss no longer commutes with operation calls.

1 Introduction

λ_C interprets terms via a family of *augmented selection monads*

$$\widehat{S}_\epsilon(X) = (X \rightarrow \widehat{R}_\epsilon) \rightarrow \widehat{W}_\epsilon(X),$$

where $\widehat{W}_\epsilon$ is an auxiliary monad of *effect trees decorated with loss* and $\widehat{R}_\epsilon = \widehat{W}_\epsilon(R)$ is the type of loss continuations, for R the (additive) monoid in which losses are recorded. Intuitively, a value of $\widehat{S}_\epsilon(X)$ takes a *loss continuation* $\gamma : X \rightarrow \widehat{R}_\epsilon$ – an estimate of the eventual loss incurred by each possible result – and produces an effect tree of type $\widehat{W}_\epsilon(X)$: an unevaluated computation that may call operations from the ambient effect signature and record loss along the way.

We build $\widehat{W}_\epsilon$ by applying the *free monad transformer* for the effect signature at level ϵ to the *writer monad* $\text{Writer}_R = R \times (-)$:

$$\widehat{W}_\epsilon(X) = \text{FreeT}_\epsilon(\text{Writer}_R)(X),$$

threading a writer layer through *every* constructor of the effect tree, so that a loss contribution is attached at every node rather than combined into a single pair at the leaves. Had loss instead been combined once at each leaf, the resulting monad would validate a commutativity law under which a recorded loss can be freely reordered past any neighbouring operation call; attaching loss at every node gives up this law, since the relative order of a **tell** and an operation call becomes part of the data of the tree.

Effects are eliminated by handlers **with** h **from** v **handle** e : h supplies one clause $op_j \mapsto \lambda^e z : \text{in}. e_j$ for each operation op_j it handles and a *return* clause; v supplies the initial loss estimate the handled body runs under. Handlers here carry no additional threaded parameter – no running accumulator carried across successive resumptions of a delimited continuation. Section 5 gives the resulting handler semantics.

We additionally restrict operation and argument types to *ground* (first-order) types: no operation may take or return a value of function type. (The object language’s *expressions* remain fully higher-order throughout – functions may be passed to and returned from handlers, and values may have arbitrary function types – only an operation’s own *out/in* signature may not itself mention $\Rightarrow$.) This keeps $\widehat{W}_\epsilon$ an ordinary, non-mutually-recursive inductive definition: with only ground argument and result types, $\widehat{W}_\epsilon$ never needs the general type semantics $S[-]$, only a ground one, so it does not need to be defined simultaneously with $S[-]$. Lifting this restriction to allow higher-order operation signatures is left for future work.

Section 2 recalls the semantics of types. Section 3 develops the monad $\widehat{W}_\epsilon$ and the resulting selection monad $\widehat{S}_\epsilon$. Section 4 gives the semantics of expressions. Section 5 gives the semantics of handlers.

2 Semantics of Types

Types σ, τ are built from base types, tuples, and function types $\sigma \rightarrow \tau ! \epsilon$ decorated with an effect context ϵ (a multiset of effect labels available to the function body). A signature Σ fixes, for each effect label ℓ , a family of operations $op : out \xrightarrow{\ell} in$ with out, in ground types. Expressions e may call these operations, record loss via `loss`(e), sequence a loss-continuation via $\blacktriangleright$ and a local construct $\langle e \rangle_g^\epsilon$, discard recorded loss via `reset`, and be handled by handlers h with clauses $op_i \mapsto \dots$ and $return \mapsto \dots$, carrying no separate parameter.

The semantics of types, $S[\sigma]$, is given by

$$S[b] = [b], \quad S[(\sigma_1, \dots, \sigma_n)] = S[\sigma_1] \times \dots \times S[\sigma_n], \quad S[\sigma \rightarrow \tau ! \epsilon] = S[\sigma] \rightarrow \widehat{S}_\epsilon(S[\tau]),$$

where $\widehat{S}_\epsilon$ is the selection-monad family developed in Section 3.

3 The Monad $\widehat{W}_\epsilon$

3.1 The carrier $\widehat{W}_\epsilon$

An ϵ -algebra is a set X with maps $\varphi_{\ell, op, i} : S[out] \times X^{S[in]} \rightarrow X$ for each $\ell \in \epsilon$, $op : out \xrightarrow{\ell} in$, $0 < i \leq \epsilon(\ell)$; the free such algebra on a set X is written $F_\epsilon(X)$. Because the writer layer here is woven through every node of an effect tree rather than paired only at its leaves, a single family $\widehat{W}_\epsilon$ suffices where a leaves-only construction would instead need two separate operators.

Definition 1 (The monad $\widehat{W}_\epsilon$). *For a set X , let $\widehat{W}_\epsilon(X)$ be the least set Y such that*

$$Y = R \times \left(X + \sum_{\substack{\ell \in \epsilon, \text{ } op : out \xrightarrow{\ell} in \\ 0 < i \leq \epsilon(\ell)}} S[out] \times Y^{S[in]} \right).$$

We write an element as a pair (r, n) with $r \in R$ (the loss recorded at this node) and n either a leaf $\iota_1(x)$ for $x \in X$, or an internal node $\iota_2((\ell, op, i), (o, \kappa))$ decorated exactly as in F_ϵ .

This is exactly the least-fixpoint reading of $\text{FreeT}_\epsilon(\text{Writer}_R)(X)$: unrolling the usual defining equation of a free monad transformer, $\text{FreeT } \Sigma M(X) \cong M(X + \Sigma(\text{FreeT } \Sigma M(X)))$, at $M = \text{Writer}_R = R \times (-)$ and Σ the ϵ -effect signature, gives exactly the equation above. (As with any

such construction, $\widehat{W}_\epsilon$ and the selection monad $\widehat{S}_\epsilon$ of Section 3.4 are mutually recursive across decreasing effect contexts; we take well-foundedness of this recursion for granted.)

We write $\widehat{R}_\epsilon := \widehat{W}_\epsilon(R)$ for the type of *loss continuations*. $\widehat{R}_\epsilon$ retains the full tree shape of $\widehat{W}_\epsilon$ (an unhandled operation can still be pending at any depth); it merely has R , rather than an arbitrary X , as its leaf-value type, so $\widehat{R}_\epsilon$ is not isomorphic in general to any simpler, non-recursive type of accumulated losses.

3.2 Free layered ϵ -algebras

Definition 2 (Layered ϵ -algebra). *A layered ϵ -algebra is a set Y equipped with an ϵ -algebra structure ψ and an additive action $\cdot : R \times Y \rightarrow Y$ ($0 \cdot y = y$, $r \cdot (s \cdot y) = (r + s) \cdot y$), with no requirement that $\cdot$ commute with ψ .*

Definition 3 (Free layered ϵ -algebra). *$\widehat{W}_\epsilon(X)$ is the free layered ϵ -algebra on X : its own structure is*

$$\widehat{\varphi}_{\ell, op, i}(o, \kappa) =_{\text{def}} (0, \iota_2((\ell, op, i), (o, \kappa))), \quad r \cdot (r_0, n) =_{\text{def}} (r + r_0, n),$$

with unit $\eta^{\widehat{W}_\epsilon}(x) = (0, \iota_1(x))$; and for any layered ϵ -algebra $(Y, \psi, \cdot)$ and function $f : X \rightarrow Y$, the unique homomorphic extension $f^{\dagger \widehat{W}_\epsilon} : \widehat{W}_\epsilon(X) \rightarrow Y$ is given by

$$f^{\dagger \widehat{W}_\epsilon}(r_0, \iota_1(x)) = r_0 \cdot f(x), \quad f^{\dagger \widehat{W}_\epsilon}(r_0, \iota_2((\ell, op, i), (o, \kappa))) = r_0 \cdot \psi_{\ell, op, i}(o, f^{\dagger \widehat{W}_\epsilon} \circ \kappa).$$

Both clauses are well defined by structural recursion since $\widehat{W}_\epsilon(X)$ is a least (hence well-founded, finitely-branching-and-depth) set. One checks directly that $f^{\dagger \widehat{W}_\epsilon} \circ \eta^{\widehat{W}_\epsilon} = f$ and that $f^{\dagger \widehat{W}_\epsilon}$ is a homomorphism of layered ϵ -algebras, using only the additive action law of Y – no commutation between ψ and $\cdot$ is needed anywhere in the argument, which is precisely why $\widehat{\varphi}$ and the action need not commute for the recursion to be well founded.

Remark 4. *Instantiating $(Y, \psi, \cdot) := (\widehat{W}_\epsilon(Z), \widehat{\varphi}, \cdot)$ for a function $f : X \rightarrow \widehat{W}_\epsilon(Z)$ turns the homomorphic-extension formula above into the Kleisli extension (*bind*) of $\widehat{W}_\epsilon$: writing $\text{let}_{\widehat{W}_\epsilon} x \in X$ be u in $f(x) := f^{\dagger \widehat{W}_\epsilon}(u)$,*

$$\text{let}_{\widehat{W}_\epsilon} x \in X \text{ be } (r_0, \iota_1(x')) \text{ in } f(x) = r_0 \cdot f(x'),$$

$$\begin{aligned} \text{let}_{\widehat{W}_\epsilon} x \in X \text{ be } (r_0, \iota_2((\ell, op, i), (o, \kappa))) \text{ in } f(x) \\ = \left(r_0, \iota_2((\ell, op, i), (o, \lambda a. \text{let}_{\widehat{W}_\epsilon} x \in X \text{ be } \kappa(a) \text{ in } f(x))) \right). \end{aligned}$$

That is: at a leaf, the node's own loss r_0 is added to whatever the continuation produces (via the action); at an internal node, r_0 is left exactly where it is and only the children are rebound. This is exactly the standard bind for a monad transformer stacked over Writer_R .

3.3 Loss, commutativity, and node-local combinators

Because a loss is recorded at the specific node at which it occurs, the order of a loss relative to a neighbouring operation is part of the data of $\widehat{W}_\epsilon(X)$: $\widehat{\varphi}$ applied first and then tell (defined below) applied to the result is a *different* element from tell applied first and then $\widehat{\varphi}$. Here we simply record the combinators we will need:

$$\text{tell}(r)(r_0, n) =_{\text{def}} (r + r_0, n) \quad (\text{i.e. } \text{tell}(r) = r \cdot (-)),$$

$$\text{censor}_0(r_0, n) =_{\text{def}} \begin{cases} (0, \iota_1(x)) & n = \iota_1(x) \\ (0, \iota_2((\ell, op, i), (o, \lambda a. \text{censor}_0(\kappa(a)))))) & n = \iota_2((\ell, op, i), (o, \kappa)), \end{cases}$$

the latter recursively zeroing every recorded loss throughout the tree while leaving its shape and leaves untouched, and

$$\Delta(r_0, n) =_{\text{def}} \begin{cases} (0, \iota_1(r_0)) & n = \iota_1(()) \\ \text{tell}(r_0)(0, \iota_2((\ell, op, i), (o, \lambda a. \Delta(\kappa(a)))))) & n = \iota_2((\ell, op, i), (o, \kappa)), \end{cases}$$

a map $\widehat{W}_\epsilon(1) \rightarrow \widehat{W}_\epsilon(R)$ that recursively *sums* all losses along each root-to-leaf path into that leaf's now R -valued payload, while zeroing every node's own recorded loss along the way (note that unlike censor_0 , Δ leaves a node's own loss r_0 in place and only ever zeroes leaves). We also need the evident generalisation to arbitrary payload type X , which pairs each leaf value with the total accumulated loss on its path rather than discarding it:

$$\begin{aligned} \text{collect}_X(r_0, \iota_1(x)) &= (0, \iota_1((x, r_0))), \\ \text{collect}_X(r_0, \iota_2((\ell, op, i), (o, \kappa))) &= \left(0, \iota_2((\ell, op, i), (o, \lambda a. \text{bump}_{r_0}(\text{collect}_X(\kappa(a))))))\right), \end{aligned}$$

where $\text{bump}_r : \widehat{W}_\epsilon(X \times R) \rightarrow \widehat{W}_\epsilon(X \times R)$ adds r into the recorded total at *every* leaf beneath the current node (needed because, unlike tell , collect_X must push a node's loss down into leaves that may be arbitrarily deep, not just combine it at the top):

$$\begin{aligned} \text{bump}_r(0, \iota_1((x, s))) &= (0, \iota_1((x, r + s))), \\ \text{bump}_r(0, \iota_2((\ell, op, i), (o, \kappa))) &= (0, \iota_2((\ell, op, i), (o, \lambda a. \text{bump}_r(\kappa(a)))))) \end{aligned}$$

(the outer loss is always 0 on the nose in the argument of bump_r , since collect_X zeroes it just before recursing). One checks $\text{bump}_0 = \text{id}$ and $\text{bump}_r \circ \text{bump}_s = \text{bump}_{r+s}$ directly from these clauses. Finally $\Delta = \widehat{W}_\epsilon(\pi_2) \circ \text{collect}_1 : \widehat{W}_\epsilon(1) \rightarrow \widehat{W}_\epsilon(R)$, i.e. Δ is collect_1 followed by forgetting the (unit-typed) leaf value and keeping only the accumulated loss.

3.4 The selection monad $\widehat{S}_\epsilon$

With $\widehat{W}_\epsilon$ and $\widehat{R}_\epsilon = \widehat{W}_\epsilon(R)$ in hand (Section 3.1), define

$$\widehat{S}_\epsilon(X) = (X \rightarrow \widehat{R}_\epsilon) \rightarrow \widehat{W}_\epsilon(X), \quad \eta^{\widehat{S}_\epsilon}(x) = \lambda \gamma. \eta^{\widehat{W}_\epsilon}(x).$$

$\widehat{R}_\epsilon$ is itself a layered ϵ -algebra (being $\widehat{W}_\epsilon(R)$), with action $r \cdot u = \text{tell}(r)(u)$. For $F \in \widehat{S}_\epsilon(Y)$ and a loss function $\gamma : Y \rightarrow \widehat{R}_\epsilon$, define

$$\widehat{\mathcal{R}}_\epsilon(F \mid \gamma) =_{\text{def}} \text{let}_{\widehat{W}_\epsilon} (a, r_1) \in Y \times R \text{ be } \text{collect}_Y(F(\gamma)) \text{ in } \widehat{W}_\epsilon(r_1 + -)(\gamma(a)) \in \widehat{R}_\epsilon.$$

Remark 5. A natural first attempt would be the plain homomorphic extension $\gamma^{\dagger \widehat{W}_\epsilon}(F(\gamma))$: it typechecks, since $\gamma : Y \rightarrow \widehat{R}_\epsilon$ and $F(\gamma) : \widehat{W}_\epsilon(Y)$. It is not, however, what we want, because $\gamma^{\dagger \widehat{W}_\epsilon}$ simply replaces each leaf value x of $F(\gamma)$ by $\gamma(x)$, discarding the loss r_1 recorded along the path to that leaf rather than combining it with $\gamma(x)$. The collect -based definition above instead extracts, at each leaf of $F(\gamma)$, the pair of that leaf's own value a and the total loss r_1 recorded along the path to it, and adds r_1 into every leaf of $\gamma(a)$ (via $\widehat{W}_\epsilon(r_1 + -)$) rather than trying to fold r_1 into $\gamma(a)$'s own root via a single top-level tell . The two coincide whenever $\gamma(a)$ happens to be a bare leaf, but not in general once $\gamma(a)$ may itself be a tree with a pending operation at arbitrary depth.

Then the Kleisli extension of $f : X \rightarrow \widehat{S}_\epsilon(Y)$ is, verbatim,

$$f^{\dagger \widehat{S}_\epsilon}(F) = \lambda \gamma \in Y \rightarrow \widehat{R}_\epsilon. \text{ let }_{\widehat{W}_\epsilon} x \in X \text{ be } F(\lambda x \in X. \widehat{R}_\epsilon(fx \mid \gamma)) \text{ in } fx \gamma,$$

and $\widehat{S}_\epsilon(X)$ is itself an ϵ -algebra via

$$\widehat{\varphi}_{\ell, op, i}^X(o, f)(\gamma) = \widehat{\varphi}_{\ell, op, i}(o, \lambda x \in in. f(x)(\gamma)).$$

All bookkeeping of losses is delegated to the fact that $\widehat{W}_\epsilon$ itself already carries R at every node: the φ used in the last equation is the one-argument $\widehat{\varphi}$ of $\widehat{W}_\epsilon(X)$ directly, and nothing in this subsection ever needs to mention $R \times -$ explicitly.

4 Semantics of Expressions

The semantics of expressions is given compositionally, largely in terms of the abstract monad interface of $\widehat{S}_\epsilon$ (η , Kleisli extension $\dagger \widehat{S}_\epsilon$, and the algebra maps $\widehat{\varphi}^X$). The clauses for variables, constants, tuples, projection, abstraction, application, and operation calls read

$$\mathbb{S}[\text{op}(e)](\rho) = \text{let}_{\widehat{S}_\epsilon} a \in \mathbb{S}[\text{out}] \text{ be } \mathbb{S}[e](\rho) \text{ in } \widehat{\varphi}_{\ell, op, \epsilon(\ell)}^X(a, (\eta^{\widehat{S}_\epsilon})^{\mathbb{S}[\text{in}]}),$$

with the remaining constructs (variables, constants, tuples, projection, abstraction, application) the evident generic clauses of a monadic semantics with algebra structure $\widehat{\varphi}^X$. The four constructs that manipulate the shape of $\widehat{W}_\epsilon$ directly – **loss**, **(►)**, the local construct $\langle e \rangle_g^\epsilon$, and **reset** – together with the auxiliary loss-function semantics $\mathcal{L}$, are given below.

loss(e). Since a bare tell is already available as a $\widehat{S}_\epsilon$ -level operation, the semantics of **loss(e)** needs no pattern matching on the shape of $\mathbb{S}[e]$:

$$\mathbb{S}[\text{loss}(e)](\rho) = \lambda \gamma. \text{ let }_{\widehat{W}_\epsilon} a \in R \text{ be } \mathbb{S}[e](\rho)(\gamma) \text{ in tell}(a)(\eta^{\widehat{W}_\epsilon}(())).$$

Loss-function semantics $\mathcal{L}$. For $\Gamma, x : \sigma \vdash e : \text{loss} ! \epsilon$, define $\mathbb{L}[\lambda^\epsilon x : \sigma. e] : \widehat{S}_\Gamma \rightarrow \widehat{S}_\sigma \rightarrow \widehat{R}_\epsilon$ by

$$\mathbb{L}[\lambda^\epsilon x : \sigma. e](\rho)(a) = \mathbb{S}[e](\rho[a/x])(\gamma_0), \quad \gamma_0 = \lambda r \in R. \eta^{\widehat{W}_\epsilon}(()).$$

Because $\widehat{R}_\epsilon = \widehat{W}_\epsilon(R)$ and $\mathbb{S}[e](\rho[a/x]) : \widehat{S}_\epsilon(R) = (R \rightarrow \widehat{W}_\epsilon(R)) \rightarrow \widehat{W}_\epsilon(R)$ for $e : \text{loss} ! \epsilon$, applying it to γ_0 already produces a value of exactly $\widehat{R}_\epsilon$.

$e_1 \blacktriangleright \lambda^{\epsilon_1} x : \sigma_1. e_2$. Run e_1 under the hypothetical continuation $\mathbb{L}[\lambda x. e_2](\rho)$, obtaining a value a together with the loss r_1 this incurred (via **collect**); then bump $\mathbb{L}[\lambda x. e_2](\rho)(a)$'s own leaves by r_1 :

$$\begin{aligned} \mathbb{S}[e_1 \blacktriangleright \lambda^{\epsilon_1} x : \sigma_1. e_2](\rho) &= \lambda \gamma. \text{ let }_{\widehat{W}_{\epsilon_1}} (a, r_1) \in \mathbb{S}[\sigma_1] \times R \text{ be} \\ &\quad \text{collect}_{\mathbb{S}[\sigma_1]}(\mathbb{S}[e_1](\rho)(\mathbb{L}[\lambda x. e_2](\rho))) \text{ in } \widehat{W}_{\epsilon_1}(r_1 + -)(\mathbb{L}[\lambda x. e_2](\rho)(a)). \end{aligned}$$

Remark 6. A two-stage decomposition – extract (a, r_1) from e_1 's evaluation, separately extract (r_3, r_2) from $\mathbb{L}[g](\rho)(a)$'s own recorded loss versus returned value via a second **collect**, then **tell**(r_2) the sum $r_1 + r_3$ – is unnecessary: since $\mathbb{L}[g](\rho)(a)$ is already the finished $\widehat{R}_{\epsilon_1}$ -value one wants (per $\mathcal{L}$'s definition above), there is nothing left to extract from it a second time – bumping every one of its leaves by r_1 (via a $\widehat{W}_{\epsilon_1}$ -map, not a second **collect/tell** round-trip) already produces the right tree whatever shape $\mathbb{L}[g](\rho)(a)$ has, including a pending stuck operation at arbitrary depth, whereas re-extracting and re-**tell**-ing would silently redistribute g 's own recorded loss down past such a stuck node. Both formulations agree when $\mathbb{L}[g](\rho)(a)$ happens to be a bare leaf.

As before, this is constant in γ , matching “ $\blacktriangleright$ disregards g ”; $\mathbb{L}[g](\rho)$ has codomain $\widehat{R}_{\epsilon_1}$ for g ’s own (possibly smaller) effect context, so the left-hand side’s ambient ϵ and g ’s own $\epsilon_1 \subseteq \epsilon$ are related by the same kind of sub-effecting inclusion (left implicit here, as throughout) that Section 5 needs for the handler formula below.

$\langle e \rangle_g^{\epsilon_1}$. This clause only substitutes a different loss continuation and is oblivious to the concrete shape of $\widehat{W}_\epsilon$:

$$\mathbb{S}[\langle e \rangle_g^{\epsilon_1}](\rho) = \lambda\gamma. \mathbb{S}[e](\rho)(\mathbb{L}[g](\rho)).$$

reset e . Run e under γ , then discard all recorded loss while keeping the value, which is exactly censor_0 from Section 3.3:

$$\mathbb{S}[\text{reset } e](\rho) = \lambda\gamma. \text{censor}_0(\mathbb{S}[e](\rho)(\gamma)).$$

5 Semantics of Handlers

Fix $\rho \in \widehat{S}_\Gamma$ and $\gamma \in \widehat{S}_{\sigma'} \rightarrow \widehat{R}_\epsilon$, and build a layered $\epsilon\ell$ -algebra structure directly on

$$A = \widehat{W}_\epsilon(\widehat{S}_{\sigma'}).$$

With no handler parameter, A carries no exponent at all – it is exactly $\widehat{W}_\epsilon(\widehat{S}_{\sigma'})$, the *same* shape as $\widehat{W}_\epsilon(X)$ ’s own native algebra from Section 3.2. For the operations *not* handled by h ($\ell_1 \in \epsilon$, i.e. shallower than the newly-introduced occurrence of ℓ), pass-through:

$$\psi_{\ell_1, op, i}(o, k) = \widehat{\varphi}_{\ell_1, op, i}(o, k).$$

(With no p to abstract over, this is $\widehat{\varphi}$ itself, applied directly to the already-recursed continuation k – there is nothing left to do beyond re-wrapping.) For the handler’s own clauses $op_j \mapsto \lambda^\epsilon z. in. e_j$, at the newly introduced occurrence ($i = \epsilon(\ell) + 1$):

$$\psi_{\ell, op_j, i}(o, k) = \mathbb{S}[e_j](\rho[(o, l_1, k_1)/z])(\gamma),$$

where the delimited continuation is $k_1(a) = k(a)$ (no p to re-thread), and the choice continuation is

$$l_1(a) = \lambda\gamma_1. \text{let}_{\widehat{W}_\epsilon} (r_1, r_2) \in R \times R \text{ be } \text{collect}_R(\gamma^\dagger \widehat{W}_\epsilon(k(a))) \text{ in } \eta^{\widehat{W}_\epsilon}(r_1 + r_2) \in \widehat{S}_\epsilon(R),$$

i.e. it *ignores* its nominal continuation γ_1 , runs the delimited continuation under the ambient γ , extends γ homomorphically over the result to obtain a value of $\widehat{W}_\epsilon(R) = \widehat{R}_\epsilon$, and then sums each leaf’s own value against the total loss recorded along the path to it – collect_R generalising plain Δ here for the same reason $\widehat{\mathcal{R}}_\epsilon$ ’s own definition does (Section 3.4): the tree $\gamma^\dagger \widehat{W}_\epsilon(k(a))$ already has genuinely R -valued leaves (from γ), not unit-valued ones, so a leaf’s own value must be *added* to the path total rather than replaced by it.

The return clause needs no multiplication by an accumulated loss either – because $\widehat{W}_\epsilon$ already threads it – and, with no p to abstract over, is simply

$$s(a) = \mathbb{S}[e_r](\rho[a/z])(\gamma) \in A.$$

Finally, using the free-algebra property of $\widehat{W}_{\epsilon\ell}(\widehat{S}_\sigma)$ over A (Section 3.2, instantiated directly at $X = \widehat{S}_\sigma$, with no $R \times$ pairing needed to seed the extension):

$$\mathbb{S}[h](\rho)(G)(\gamma) = s^\dagger \widehat{W}_{\epsilon\ell} \left(G(\lambda a \in \widehat{S}_\sigma. \widehat{\mathcal{R}}_\epsilon(s(a) \mid \gamma)) \right),$$

where the loss fed to G 's own continuation is $\widehat{\mathcal{R}}_\epsilon(s(a) \mid \gamma) - \widehat{\mathcal{R}}_\epsilon$ from Section 3.4, *not* a plain $\gamma^{\dagger\widehat{W}_\epsilon}(s(a))$, since $\widehat{\mathcal{R}}_\epsilon$ is itself collect-based rather than a plain homomorphic extension: $s(a)$ is a selection function $\widehat{S}_\epsilon(\widehat{S}_{\sigma'})$, not a bare tree, so it is $\widehat{\mathcal{R}}_\epsilon$ and not $\dagger\widehat{W}_\epsilon$ that applies to it in the first place – and the inclusion $\widehat{R}_\epsilon \hookrightarrow \widehat{R}_{\epsilon\ell}$ (from $\epsilon \subseteq \epsilon\ell$) is left implicit, exactly as sub-effect inclusions are left implicit throughout.